Trivia Colombia – Juego de Preguntas

Parcial – Proyecto de Desarrollo Full Stack

Dato	Información
Estudiante	[Nombre del estudiante]
Institución	Universidad de Santander (UDES)
Asignatura	Desarrollo Full Stack
Docente	Carlos Carrascal
Tipo de entrega	Parcial basado en proyecto
Fecha	Octubre de 2026

1. Descripción del Proyecto

Trivia Colombia es un juego web de preguntas y respuestas sobre Colombia, desarrollado con HTML5, CSS3 y JavaScript. El jugador escribe su nombre, elige una categoría (Geografía, Historia, Cultura y Tradiciones, Deportes o Ciencia y Naturaleza) y un nivel de dificultad, y responde una ronda de 10 preguntas de selección múltiple con tiempo límite. Cada respuesta correcta suma puntos según la rapidez y, al final, el resultado se guarda en un ranking local.

El proyecto inicia con el diseño de la interfaz en Figma o MockFlow: primero se elaboran los wireframes y el prototipo de todas las pantallas, y luego se construye la aplicación respetando ese diseño. El juego funciona completamente en el navegador; el banco de preguntas se guarda en un archivo del proyecto y el ranking y las estadísticas se almacenan con localStorage, sin necesidad de servidor ni inicio de sesión.

2. Objetivos

2.1 Objetivo General

Diseñar y desarrollar un juego web de trivia sobre Colombia, partiendo de un prototipo elaborado en Figma o MockFlow, y aplicando HTML5, CSS3 y JavaScript para construir una aplicación interactiva, responsive y desplegada en internet.

2.2 Objetivos Específicos

- Diseñar los wireframes, los mockups y un prototipo navegable de todas las pantallas del juego en Figma o MockFlow, en versión escritorio y móvil.
- Definir una guía de estilos (colores, tipografías, botones e íconos) y aplicarla de forma consistente en el desarrollo.
- Construir la estructura de las pantallas con etiquetas semánticas de HTML5.
- Maquetar una interfaz responsive con CSS3 usando Flexbox, Grid, variables y animaciones.
- Implementar la lógica del juego en JavaScript: selección aleatoria de preguntas, temporizador, validación de respuestas, comodines y puntaje.
- Organizar un banco de al menos 50 preguntas en un archivo de datos separado del código de la lógica.

- Guardar el ranking y las estadísticas del jugador con localStorage.
- Publicar el código en GitHub y desplegar el juego en Vercel.

3. Problema a Resolver y Justificación

Muchos estudiantes conocen poco sobre la geografía, la historia y la cultura de Colombia, y las formas tradicionales de estudiar estos temas suelen resultar poco atractivas. Por otro lado, muchos juegos de preguntas disponibles en internet tienen publicidad invasiva, no están enfocados en temas colombianos o no funcionan bien en dispositivos móviles.

Trivia Colombia propone una alternativa ligera, gratuita y educativa que se ejecuta directamente en el navegador. Además, el proyecto permite al estudiante recorrer el proceso completo de creación de un producto web: diseño de la experiencia de usuario en una herramienta profesional de prototipado, desarrollo del frontend, control de versiones y despliegue en la nube, sentando las bases para incorporar más adelante un backend en la asignatura.

4. Alcance

4.1 Incluye

- Diseño completo de la interfaz en Figma o MockFlow (wireframes, mockups y prototipo navegable).
- Juego para un solo jugador que se identifica solo con su nombre.
- Rondas de 10 preguntas por categoría y dificultad, con temporizador de 20 segundos por pregunta.
- Comodines: 50/50 y tiempo extra (uno de cada uno por partida).
- Retroalimentación inmediata con la respuesta correcta y una breve explicación.
- Ranking local (top 10) y estadísticas guardadas en localStorage.
- Diseño responsive para celular, tableta y computador.
- Despliegue en Vercel.

4.2 No incluye

- Registro de usuarios ni inicio de sesión.
- Servidor, API o base de datos externa.
- Ranking en línea compartido entre dispositivos.
- Modo multijugador.

5. Tecnologías y Herramientas

Tecnología / Herramienta	Uso dentro del proyecto
Figma o MockFlow	Diseño de wireframes, mockups, guía de estilos y prototipo navegable.
HTML5	Estructura semántica de las pantallas (header, main, section, footer).
CSS3	Estilos, variables de color, Flexbox, Grid, media queries y animaciones.
JavaScript (ES6+)	Lógica del juego, manipulación del DOM, eventos, temporizador, arreglos y objetos.

Tecnología / Herramienta	Uso dentro del proyecto
localStorage	Almacenamiento del ranking, las estadísticas y las preferencias del jugador.
Git y GitHub	Control de versiones y alojamiento del código en un repositorio público.
Vercel	Despliegue del juego con una URL pública y actualización automática desde GitHub.
Visual Studio Code	Editor de código con la extensión Live Server para pruebas locales.

6. Reglas del Juego

1. El jugador escribe su nombre en la pantalla de inicio.
2. Elige una categoría (o «Todas») y un nivel de dificultad: fácil, medio o difícil.
3. El juego selecciona al azar 10 preguntas que cumplan los filtros, sin repetir ninguna en la misma partida.
4. Cada pregunta tiene 4 opciones y solo una es correcta. El orden de las opciones también se mezcla al azar.
5. El jugador tiene 20 segundos para responder; si se agota el tiempo, la pregunta cuenta como incorrecta.
6. Al responder, la opción elegida se marca en verde o rojo, se resalta la correcta y se muestra una breve explicación.
7. Comodín 50/50: elimina dos opciones incorrectas. Comodín de tiempo: suma 10 segundos. Cada uno se puede usar una vez por partida.
8. Al terminar las 10 preguntas se muestra el resumen y el puntaje se guarda en el ranking local.

6.1 Sistema de Puntuación

Concepto	Puntos
Respuesta correcta	+100
Bonificación por rapidez	+5 por cada segundo restante (máximo +100)
Racha de 3 respuestas correctas seguidas	+50 adicionales
Respuesta incorrecta o sin responder	0
Uso de un comodín	-30
Multiplicador por dificultad	Fácil x1, Medio x1.5, Difícil x2

6.2 Banco de Preguntas

El banco debe tener mínimo 50 preguntas, 10 por categoría, distribuidas entre los tres niveles de dificultad, y guardarse en el archivo js/preguntas.js. Cada pregunta incluye su enunciado, cuatro opciones, la opción correcta y una explicación corta. Ejemplos:

Categoría	Pregunta	Respuesta correcta	Nivel
Geografía	¿Cuál es la capital del departamento de Santander?	Bucaramanga	Fácil
Geografía	¿Cuál es el principal río de Colombia, que la recorre de sur a norte?	Río Magdalena	Medio
Historia	¿En qué año se firmó el Acta de Independencia de Colombia?	1810	Fácil
Cultura y Tradiciones	¿En qué ciudad se celebra el carnaval declarado Patrimonio de la Humanidad por la UNESCO?	Barranquilla	Medio
Deportes	¿Qué ciclista colombiano ganó el Tour de Francia en 2019?	Egan Bernal	Medio
Ciencia y Naturaleza	¿Cuál es el ave nacional de Colombia?	Cóndor de los Andes	Fácil

7. Diseño de la Interfaz en Figma o MockFlow

Antes de escribir código, el estudiante debe diseñar la interfaz completa del juego en Figma o en MockFlow. El diseño es un entregable obligatorio y la aplicación final debe corresponder visualmente a él.

7.1 Qué debe contener el diseño

- **Wireframes de baja fidelidad:** bocetos en escala de grises de cada pantalla que muestren la distribución de los elementos.
- **Mockups de alta fidelidad:** las mismas pantallas con colores, tipografías, íconos e imágenes definitivas.
- **Versión escritorio y móvil:** cada pantalla debe diseñarse al menos en dos tamaños (por ejemplo, 1440 px y 375 px de ancho).
- **Guía de estilos:** paleta de colores con sus códigos, tipografías y tamaños, y estados de los botones (normal, hover, correcto, incorrecto, deshabilitado).
- **Prototipo navegable:** pantallas conectadas entre sí para simular el recorrido del jugador desde el inicio hasta el resultado.

7.2 Pantallas mínimas

Pantalla	Elementos principales
Inicio	Logo y nombre del juego, campo para el nombre del jugador, botones «Jugar», «Instrucciones» y «Ranking».
Instrucciones	Reglas del juego, explicación de los comodines y del sistema de puntuación.
Configuración de partida	Tarjetas de las 5 categorías con ícono, opción «Todas» y selector de dificultad.
Juego	Enunciado, cuatro botones de opción, temporizador, barra de progreso (1/10), puntaje actual y botones de comodines.

Pantalla	Elementos principales
Retroalimentación	Estado de la pregunta respondida: opción correcta en verde, incorrecta en rojo y explicación.
Resultado	Puntaje final, número de aciertos, tiempo total, mensaje según el desempeño y botones «Jugar de nuevo» y «Ver ranking».
Ranking y estadísticas	Top 10 de puntajes (nombre, puntaje, categoría y fecha) y estadísticas del jugador.

7.3 Guía de estilos sugerida

Paleta de colores: inspirada en la bandera de Colombia: amarillo (#FCD116) para acentos, azul (#003893) para encabezados y botones, rojo (#CE1126) para errores y alertas, verde (#2E9E5B) para aciertos y fondos claros para mantener la legibilidad.

Tipografía y componentes: una fuente sans-serif legible (por ejemplo, Poppins o Montserrat de Google Fonts), botones con un tamaño mínimo de 44 px de alto para pantallas táctiles, esquinas redondeadas y contraste suficiente entre texto y fondo.

8. Estructura del Proyecto

El repositorio debe organizarse de la siguiente manera:

```
trivia-colombia/
├── index.html              → Estructura de todas las pantallas
├── css/
│   ├── styles.css          → Estilos generales y variables
│   └── responsive.css      → Media queries
├── js/
│   ├── app.js              → Punto de entrada y navegación
│   ├── juego.js            → Lógica: preguntas, tiempo, puntaje
│   ├── preguntas.js        → Banco de preguntas
│   ├── ui.js               → Renderizado de pantallas en el DOM
│   └── almacenamiento.js   → Ranking y estadísticas (localStorage)
├── assets/
│   ├── img/                → Logo, íconos de categorías
│   └── sonidos/            → Efectos de acierto y error
├── docs/
│   └── diseno/             → Capturas o PDF del diseño en Figma/MockFlow
└── README.md
```

9. Funcionalidades

Módulo	Funcionalidades
Inicio	Ingreso del nombre del jugador (obligatorio) y navegación a instrucciones, juego y ranking.
Configuración de partida	Selección de categoría y dificultad; muestra el mejor puntaje del jugador en esa combinación.
Juego	Pregunta con 4 opciones mezcladas, temporizador, barra de progreso, comodines y retroalimentación inmediata.

Módulo	Funcionalidades
Resultado	Puntaje total, aciertos, tiempo empleado, posición en el ranking y revisión de las preguntas falladas.
Ranking	Top 10 local con nombre, puntaje, categoría y fecha; opción para borrar el ranking.
Estadísticas	Partidas jugadas, porcentaje de aciertos, mejor puntaje y categoría con mejor desempeño.
Configuración	Activar o desactivar el sonido y cambiar entre tema claro y oscuro.

10. Requerimientos

10.1 Requerimientos Funcionales

Código	Requerimiento	Prioridad
RF-01	El sistema debe solicitar el nombre del jugador antes de iniciar y no permitir un nombre vacío.	Alta
RF-02	El sistema debe permitir elegir categoría y dificultad.	Alta
RF-03	El sistema debe seleccionar 10 preguntas aleatorias sin repetir dentro de la partida.	Alta
RF-04	El sistema debe mezclar el orden de las opciones de cada pregunta.	Media
RF-05	El sistema debe mostrar un temporizador de 20 segundos por pregunta.	Alta
RF-06	El sistema debe validar la respuesta, indicar si fue correcta y mostrar la explicación.	Alta
RF-07	El sistema debe bloquear las opciones una vez respondida la pregunta.	Alta
RF-08	El sistema debe calcular el puntaje según las reglas de la sección 6.1.	Alta
RF-09	El sistema debe ofrecer los comodines 50/50 y tiempo extra, uno de cada uno por partida.	Media
RF-10	El sistema debe mostrar una barra de progreso con la pregunta actual.	Media
RF-11	El sistema debe mostrar el resumen de la partida al finalizar.	Alta
RF-12	El sistema debe guardar el ranking y las estadísticas en localStorage.	Alta
RF-13	El sistema debe mostrar el top 10 del ranking ordenado de mayor a menor puntaje.	Media
RF-14	El sistema debe permitir jugar de nuevo o volver al inicio.	Alta

10.2 Requerimientos No Funcionales

Código	Categoría	Requerimiento
RNF-01	Diseño	La aplicación debe corresponder al diseño elaborado en Figma o MockFlow.

Código	Categoría	Requerimiento
RNF-02	Usabilidad	La navegación debe ser intuitiva; un usuario nuevo debe poder jugar sin ayuda.
RNF-03	Responsive	La interfaz debe adaptarse a pantallas desde 360 px hasta escritorio.
RNF-04	Rendimiento	La respuesta a cada acción del jugador debe ser inmediata.
RNF-05	Compatibilidad	Debe funcionar en Chrome, Firefox, Edge y Safari en sus versiones actuales.
RNF-06	Accesibilidad	Contraste adecuado, textos alternativos en imágenes y botones operables con teclado.
RNF-07	Mantenibilidad	Código modular, comentado, con nombres descriptivos y los datos separados de la lógica.
RNF-08	Disponibilidad	El juego debe estar desplegado en Vercel y accesible mediante una URL pública.

11. Historias de Usuario

ID	Como...	Quiero...	Para...
HU-01	jugador	escribir mi nombre al empezar	aparecer en el ranking.
HU-02	jugador	elegir la categoría y la dificultad	jugar sobre los temas que me interesan.
HU-03	jugador	saber de inmediato si acerté y por qué	aprender de mis errores.
HU-04	jugador	usar comodines	tener una ayuda en preguntas difíciles.
HU-05	jugador	ver el ranking	comparar mis puntajes con los de otras personas que jueguen en el mismo dispositivo.
HU-06	jugador	jugar desde mi celular	practicar en cualquier lugar.
HU-07	docente	usar el juego en clase	repasar temas de Colombia de forma entretenida.

12. Flujo del Juego

1. Inicio: el jugador abre la aplicación y escribe su nombre.
2. Configuración: elige categoría y dificultad y presiona «Comenzar».
3. Preparación: el juego filtra el banco, mezcla las preguntas, toma 10 y reinicia el puntaje.
4. Pregunta: se muestra el enunciado con sus opciones y arranca el temporizador.
5. Respuesta: el jugador elige una opción (o se agota el tiempo); se valida, se muestra la retroalimentación y se actualiza el puntaje.
6. Avance: si quedan preguntas, se pasa a la siguiente (paso 4); si no, termina la partida.

7. Resultado: se muestra el resumen y se guardan el ranking y las estadísticas en localStorage.
8. Fin: el jugador decide jugar otra vez, ver el ranking o volver al inicio.

13. Modelo de Datos

Los datos del juego se representan con objetos y arreglos de JavaScript:

```
// Banco de preguntas (preguntas.js)
const PREGUNTAS = [
  {
    id: 1,
    categoria: 'naturaleza',
    dificultad: 'facil',
    enunciado: '¿Cuál es el ave nacional de Colombia?',
    opciones: ['Guacamaya', 'Cóndor de los Andes', 'Colibrí', 'Tucán'],
    correcta: 1,
    explicacion: 'El cóndor de los Andes aparece en el escudo nacional.'
  }
  // ... mínimo 50 preguntas
];

// Estado de la partida (juego.js)
const estado = {
  jugador: 'Ana',
  categoria: 'naturaleza',
  dificultad: 'facil',
  preguntas: [],        // las 10 preguntas seleccionadas
  indiceActual: 0,
  aciertos: 0,
  racha: 0,
  puntaje: 0,
  tiempoRestante: 20,
  comodines: { cincuenta: true, tiempo: true }
};
```

El ranking se guarda en localStorage bajo la clave «trivia_ranking» como un arreglo de objetos { nombre, puntaje, categoria, dificultad, fecha }, y las estadísticas bajo «trivia_stats».

14. Funciones Principales

Función	Archivo	Descripción
iniciarPartida(categoria, dificultad)	juego.js	Reinicia el estado, selecciona las preguntas y muestra la primera.
obtenerPreguntas(categoria, dificultad)	juego.js	Filtra el banco, lo mezcla y devuelve 10 preguntas.
mezclar(arreglo)	juego.js	Ordena un arreglo al azar (algoritmo de Fisher-Yates).
iniciarTemporizador()	juego.js	Controla la cuenta regresiva con setInterval y detecta el tiempo agotado.
responder(indiceOpcion)	juego.js	Valida la respuesta, calcula los puntos y actualiza la racha.

Función	Archivo	Descripción
usarComodin(tipo)	juego.js	Aplica el 50/50 o el tiempo extra y descuenta puntos.
mostrarPregunta() / mostrarResultado()	ui.js	Renderizan las pantallas en el DOM.
guardarEnRanking(partida)	almacenamiento.js	Agrega la partida, ordena y conserva el top 10.

15. Plan de Pruebas

ID	Caso de prueba	Resultado esperado
CP-01	Intentar iniciar sin escribir el nombre.	Se muestra un mensaje y no se avanza.
CP-02	Iniciar partida de Historia en nivel fácil.	Aparecen 10 preguntas distintas de esa categoría y nivel.
CP-03	Jugar dos partidas seguidas con los mismos filtros.	El orden de las preguntas y de las opciones cambia.
CP-04	Responder correctamente con 12 segundos restantes.	Se suman 100 + 60 puntos (antes del multiplicador).
CP-05	Responder incorrectamente.	La opción se marca en rojo, la correcta en verde y aparece la explicación.
CP-06	Dejar agotar el tiempo.	La pregunta cuenta como incorrecta con 0 puntos.
CP-07	Hacer clic en otra opción después de responder.	No ocurre nada; las opciones están bloqueadas.
CP-08	Usar el comodín 50/50.	Desaparecen dos opciones incorrectas y el botón queda deshabilitado.
CP-09	Acertar 3 preguntas seguidas.	Se suma la bonificación de racha de +50.
CP-10	Terminar una partida.	Se muestra el resumen y el puntaje aparece en el ranking.
CP-11	Recargar la página.	El ranking y las estadísticas se conservan.
CP-12	Comparar la aplicación con el diseño en Figma o MockFlow.	Las pantallas, colores y componentes coinciden con el diseño.
CP-13	Abrir el juego en un celular de 360 px.	Todos los elementos se ven y funcionan sin desplazamiento horizontal.

16. Cronograma de Desarrollo

Semana	Actividades	Entregable
1	Análisis, requerimientos, wireframes y guía de estilos en Figma o MockFlow.	Wireframes y guía de estilos

Semana	Actividades	Entregable
2	Mockups de alta fidelidad (escritorio y móvil) y prototipo navegable. Redacción del banco de preguntas.	Diseño completo y banco de preguntas
3	Creación del repositorio y maquetación HTML y CSS de todas las pantallas.	Interfaz estática responsive
4	Lógica principal: selección de preguntas, temporizador, validación y puntaje.	Juego jugable básico
5	Comodines, ranking y estadísticas con localStorage, sonidos y tema oscuro.	Versión completa
6	Pruebas, ajustes frente al diseño, README, despliegue en Vercel y grabación del video.	Entrega final

17. Entregables del Parcial

Para la entrega final, el estudiante debe presentar los siguientes cuatro elementos. La falta de cualquiera de ellos se considera una entrega incompleta.

17.1 Diseño en Figma o MockFlow

- Enlace al archivo de Figma o MockFlow con permiso de visualización para el docente.
- El diseño debe incluir todo lo solicitado en la sección 7: wireframes, mockups de escritorio y móvil, guía de estilos y prototipo navegable.
- Exportar las pantallas (PDF o imágenes PNG) y guardarlas en la carpeta docs/diseno del repositorio.

17.2 Código fuente en GitHub

- Repositorio público en GitHub con la estructura de la sección 8.
- Historial con commits frecuentes y mensajes descriptivos que evidencien el desarrollo progresivo. No se acepta un único commit con todo el proyecto.
- README.md con: descripción, tecnologías, instrucciones para ejecutarlo localmente, capturas de pantalla y enlaces al diseño, al despliegue y al video.

17.3 Despliegue en Vercel

- El juego debe estar desplegado en Vercel con una URL pública (por ejemplo: https://trivia-colombia.vercel.app).
- Debe funcionar correctamente, sin errores en la consola del navegador, y corresponder a la última versión del repositorio.
- El enlace debe permanecer activo hasta la publicación de las notas.

17.4 Video explicativo (máximo 5 minutos)

El estudiante debe grabar un video de máximo 5 minutos explicando el desarrollo del proyecto. El video puede subirse a YouTube (como no listado) o a Google Drive con acceso público, y su enlace debe incluirse en el README.

Requisito obligatorio: el estudiante debe aparecer mostrando su rostro con la cámara encendida durante todo el video (por ejemplo, en una ventana pequeña en una esquina mientras comparte

pantalla) y explicar con su propia voz. No se aceptan videos sin cámara, con voz generada por inteligencia artificial o narrados por otra persona.

Se sugiere la siguiente distribución del tiempo:

Tiempo	Contenido
0:00 – 0:30	Presentación del estudiante en cámara (rostro visible), nombre del proyecto y objetivo.
0:30 – 1:30	Recorrido por el diseño en Figma o MockFlow: guía de estilos, pantallas y prototipo navegable.
1:30 – 2:30	Demostración del juego desplegado en Vercel: una partida completa, uso de comodines y ranking.
2:30 – 4:00	Explicación del código: banco de preguntas, selección aleatoria, temporizador, validación, puntaje y localStorage.
4:00 – 4:30	Recorrido por el repositorio en GitHub y el historial de commits.
4:30 – 5:00	Dificultades encontradas, cómo se resolvieron y conclusiones.

17.5 Forma de entrega

Enviar a través de la plataforma indicada por el docente un mensaje o documento con los siguientes enlaces:

Entregable	Enlace
Diseño (Figma o MockFlow)	https://www.figma.com/... o https://app.mockflow.com/...
Repositorio en GitHub	https://github.com/usuario/trivia-colombia
Despliegue en Vercel	https://trivia-colombia.vercel.app
Video explicativo	https://youtu.be/...

18. Mejoras Futuras

- Conectar el juego a un backend con una API y una base de datos para tener un ranking en línea compartido.
- Agregar registro e inicio de sesión de usuarios.
- Panel para que el docente agregue y edite preguntas sin modificar el código.
- Retos diarios con las mismas preguntas para todos los jugadores.
- Convertir la aplicación en una PWA instalable en el celular.

19. Conclusión

Trivia Colombia permite recorrer el proceso completo de creación de un producto web: desde el diseño de la experiencia de usuario en Figma o MockFlow, pasando por la construcción de la interfaz con HTML5 y CSS3 y la lógica interactiva con JavaScript, hasta el control de versiones en GitHub y el despliegue en Vercel. Su código organizado y sus datos separados de la lógica dejan el proyecto preparado para incorporar, en etapas posteriores de la asignatura, un backend con API, base de datos e inicio de sesión.